1. Difference Between High-Level Design (HLD) and Low-Level Design (LLD)

Understanding the boundary between HLD and LLD is the first step in system design.

- **High-Level Design (HLD):**
    - **Focus:** The "big picture" or macro-level architecture of the entire system.
    - **Scope:** Defines the overall system architecture, database selection, network topology, and how different microservices/modules interact with each other.
    - **Deliverables:** Architecture block diagrams, data flow diagrams, and technology stack choices (e.g., "We will use Kafka for messaging and PostgreSQL for user data").
    - **Target Audience:** System architects, engineering managers, and stakeholders.
- **Low-Level Design (LLD):**
    - **Focus:** The "micro-level" or component-level design.
    - **Scope:** Defines the internal logic of individual components, including class structures, interfaces, functions, data structures, and algorithms.
    - **Deliverables:** Class diagrams, sequence diagrams, state diagrams, and actual code/pseudocode.
    - **Target Audience:** Software developers who will write the actual implementation.
- **The Golden Rule: HLD is about *what* the system does and *how* components talk to each other. LLD is about *how* a specific component is built internally.**

---

2. The Purpose of HLD in System Design Interviews

Many candidates fail because they treat the interview like a coding test. HLD interviews test a completely different skill set.

- **Evaluating Trade-offs, Not Coding:** Interviewers do not want to see you write Python or Java. They want to know *why* you chose a specific technology. (e.g., *"Why did you choose Cassandra over PostgreSQL for this specific use case?"*). Every design choice must be justified.
- **Assessing Architectural Thinking:** Can you take a massive, ambiguous, real-world problem (like "Design ChatGPT") and break it down into manageable, scalable, and logical components?

- **Testing Communication Skills:** Can you clearly explain your thought process, draw a coherent diagram on a whiteboard (or Excalidraw), and defend your choices against the interviewer's probing questions?
- **Identifying Bottlenecks Early:** Do you proactively think about single points of failure, latency issues, and scalability limits before they happen in production?

---

3. The Standard 4-Step Interview Framework

Never jump straight into drawing boxes. Follow this structured 4-step framework to keep the interview organized and ensure you cover all bases.

- **Step 1: Requirements Clarification (3–5 mins)**
    - **Goal:** Understand exactly what to build.
    - **Action:** Ask questions to define **Functional Requirements** (What features? e.g., *"Users can upload a PDF and ask questions about it"*) and **Non-Functional Requirements** (Scale, latency, availability. e.g., *"We need to support 10,000 concurrent users with < 2 seconds latency"*).
- **Step 2: Estimation / Back-of-the-Envelope (3–5 mins)**
    - **Goal:** Determine the scale of the system to justify your design choices.
    - **Action:** Calculate rough numbers for **Traffic** (Requests per second), **Storage** (Data volume over 5 years), and **Bandwidth/Memory**. *Note: In AI roles, you should also estimate GPU compute needs and vector storage size here.*
- **Step 3: High-Level Design (10–15 mins)**
    - **Goal:** Draw the core architecture.
    - **Action:** Draw the main components (Client → Load Balancer → API Gateway → Core Services → Databases/Vector DBs). Define the data flow. Show how data moves from ingestion to storage to retrieval.
- **Step 4: Deep Dive (10–15 mins)**
    - **Goal:** Drill down into the most critical, complex, or bottleneck-prone parts of your design.
    - **Action:** The interviewer will usually guide this. E.g., *"How do you handle a sudden spike in RAG queries?"* or *"How do you ensure the vector DB doesn't become a*

bottleneck?" Focus on scaling, fault tolerance, caching, and specific optimizations.

---

4. AI Context: Why HLD Matters for AI Engineers

As an AI Engineer, your HLD interview will look slightly different from a traditional backend engineer's interview. Here is how you should frame your mindset:

- **Beyond the Python Script:** Anyone can write a 50-line script using LangChain to build a basic RAG app on their laptop. HLD proves you can build a **production-grade, enterprise system** that doesn't crash, leak data, or bankrupt the company under heavy load.
- **Handling Heavy Compute (GPUs):** AI workloads are incredibly resource-intensive. HLD is where you design how to provision, scale, and load-balance GPU clusters (e.g., using Kubernetes with GPU operators, vLLM for batching, or managed services like AWS Bedrock).
- **Managing Massive Data:** AI models require terabytes of data. HLD is where you design the data ingestion pipelines, feature stores, and vector databases (like Pinecone, OpenSearch, or pgvector) to handle this scale efficiently without timing out.
- **Optimizing for Latency and Cost:** LLM inference is notoriously slow and expensive. HLD allows you to design **semantic caching**, **asynchronous processing**, and **model routing** (e.g., routing simple queries to a cheap, fast model like Llama-3-8B, and complex reasoning tasks to an expensive model like GPT-4) to optimize both user experience and cloud bills.

💡 **Interview Pro-Tip for this Topic:**

If the interviewer asks, *"What is the most important thing in system design?"*, your answer should be: **"Understanding the requirements and making justified trade-offs. There is no perfect architecture, only the right architecture for the specific constraints (scale, budget, latency) of the problem."**

******

****

1. Standard Architectural Diagram Symbols

In system design interviews, your diagram is your primary communication tool. Using standard, universally understood symbols prevents confusion and shows professionalism.

- **Rectangles / Boxes:** Represent applications, services, or microservices (e.g., "API Gateway", "Chat Service", "Embedding Service").
- **Cylinders:** Represent databases or persistent storage (e.g., "PostgreSQL", "Redis"). *Pro-tip: For AI, you can use a cylinder with a small "vector" icon or just clearly label it "Vector DB (pgvector)".*
- **Clouds:** Represent external third-party services or managed cloud infrastructure (e.g., "AWS S3", "OpenAI API", "CDN").
- **Arrows:** Represent the flow of data or requests.
    - *Always label your arrows* with the protocol or data type (e.g., HTTP/JSON, gRPC, Kafka Stream, Embedding Vector).
    - Use single-headed arrows for request/response and double-headed arrows for bidirectional communication (like WebSockets).
- **Stick Figures:** Represent actors or users (e.g., "End User", "Admin", "External Data Source").

---

2. The C4 Model (Focus on Container Level)

The C4 Model is a hierarchical way to visualize software architecture. While it has four levels, **the Container level is the absolute sweet spot for HLD interviews.**

- **Level 1: System Context (The 10,000-foot view)**
    - Shows the entire system as a single box and how it interacts with users and external systems.
    - *Interview Use:* Great for the first 30 seconds to set the stage, but don't linger here.
- **Level 2: Container Diagram (THE SWEET SPOT for HLD)**
    - **Definition:** A "container" in C4 is a separately runnable/deployable unit (e.g., a web app, a mobile app, a microservice, a database). It is *not* a Docker container (though they often map 1:1).

    - **What to show:** The major building blocks, the technology choices for each, and how they communicate.
    - **AI Context Example:**
        - Client App → (HTTPS) → API Gateway → (gRPC) → RAG Orchestrator Service
        - RAG Orchestrator → (HTTP) → Embedding Model Service
        - RAG Orchestrator → (gRPC) → Vector DB (Pinecone/pgvector)
        - RAG Orchestrator → (HTTPS) → LLM Gateway → External LLM (Bedrock)
- **Level 3: Component Diagram (Deep Dive)**
    - Zooms into a *single* container to show its internal modules (e.g., inside the "RAG Orchestrator", showing the "Chunking Module", "Retrieval Module", and "Prompt Builder").
    - *Interview Use:* Use this only when the interviewer asks you to "deep dive" into a specific bottleneck.
- **Level 4: Code/Class Diagram**
    - *Interview Use:* **Ignore this.** This is Low-Level Design (LLD) and is rarely asked in HLD interviews.

---

3. Best Practices for Drawing During an Interview

Whether you are using a physical whiteboard or a virtual tool like **Excalidraw** or **Draw.io**, follow these rules to maximize your score:

- **Start Simple, Then Iterate:** Do not draw the entire complex system at once. Start with the happy path (Client → API → DB), get the interviewer's nod of approval, and then add complexity (Caching, Message Queues, Vector DBs).
- **Talk While You Draw:** Silence is your enemy. Narrate your thought process: *"I'm placing a Load Balancer here because we expect high traffic, and I'm using a Vector DB here to store the document embeddings."*
- **Label Everything Explicitly:** A box labeled "Database" is weak. A box labeled "PostgreSQL (User Metadata)" and another labeled "pgvector (Document Embeddings)" is strong.

- **Indicate Data Flow Direction:** Always use clear arrowheads. If data flows both ways, use a double-headed arrow or two separate arrows labeled "Query" and "Response".
- **Use Color Sparingly but Strategically:** Use one color (e.g., blue) for your core services, another (e.g., green) for databases, and a third (e.g., orange) for external/AI services (like the LLM Gateway). This makes the diagram readable at a glance.
- **Group Related Components:** Draw a dashed boundary box around related services and label it (e.g., "AI Inference Cluster" or "Data Ingestion Pipeline"). This shows you understand bounded contexts.

---

4. AI Context: Why This Matters for AI Engineers

In traditional backend interviews, you draw boxes for "Web Server" and "SQL DB". In an **AI Engineer interview**, your diagram must explicitly call out AI-specific components to show domain expertise:

- **Explicit AI Boxes:** You must have distinct boxes for the **Embedding Model** (e.g., "Sentence Transformers"), the **Vector Database** (e.g., "Milvus / pgvector"), and the **LLM Gateway** (handling rate limiting, fallback, and routing).
- **Highlight the Data Payload:** On your arrows, specify what is being passed. Instead of just "Request", label it "User Prompt + Retrieved Chunks (JSON)" or "Dense Vector (1536-dim)". This proves you understand the actual mechanics of the system.
- **Separate Control Plane vs. Data Plane:** In advanced AI designs, it's impressive to separate the "Ingestion Pipeline" (asynchronous, batch processing documents into vectors) from the "Query Pipeline" (synchronous, low-latency user chat). Drawing these as two distinct flows shows senior-level thinking.

💡 **Interview Pro-Tip for this Topic:**

Before your interview, **create a personal "stencil" or template** in Excalidraw. Pre-draw and save clean, good-looking icons for: *Client, API Gateway, Microservice, PostgreSQL, Redis, Kafka, Vector DB, and an LLM Cloud*. During the interview, you can drag and drop these instead of drawing them from scratch, saving you 3–5 minutes of valuable time to spend on actual design and trade-off discussions.

*****

****

What are Functional Requirements?

Functional Requirements define **what** the system must do. They describe the specific behaviors, features, and actions the system must support to satisfy the user's needs. In an interview, this is where you turn a vague, one-sentence prompt (e.g., *"Design ChatGPT"*) into a concrete, scoped problem you can actually solve in 45 minutes.

---

1. How to Gather and Clarify Requirements (Asking the Right Questions)

Never start drawing or designing immediately. Spend the first **3–5 minutes** asking targeted questions to narrow the scope. Use a "divergent then convergent" approach: ask broadly, then lock down specific constraints.

- **Clarify the User Persona (Who):**
    - *"Who are the primary users?"* (e.g., Internal employees, public consumers, enterprise clients?)
    - *"Are there different user roles?"* (e.g., Admins who upload documents vs. end-users who only query).
- **Clarify the Core Action (What):**
    - *"What is the primary action the user takes?"* (e.g., Upload a file, type a prompt, receive a streamed response).
    - *"What is the expected output?"* (e.g., Text, JSON, an image, a summarized report).
- **Clarify the Scope and Scale (How Much):**
    - *"Are we focusing on the read path (querying) or the write path (ingesting data), or both?"* (Crucial for AI: ingestion is async/batch, querying is sync/real-time).
    - *"Should we support multiple languages or just English?"*
- **Explicitly State Out-of-Scope Items:**
    - *"To keep our discussion focused, I will assume we are not building a fine-tuning pipeline or a billing system in this interview, unless you'd like me to."* (This shows maturity and time management).

---

2. Defining the Core Use Cases

Once you have clarity, explicitly list the use cases. Group them into **Must-Have (MVP)** and **Nice-to-Have (V2)** to show you can prioritize.

Example: Designing a RAG-based Document Q&A System

- **Must-Have (In Scope for Interview):**
    - User can upload a text-based document (PDF, TXT).
    - System asynchronously processes and chunks the document.
    - User can ask a natural language question about the uploaded document.
    - System returns a text answer with citations (source chunks).
- **Nice-to-Have (Out of Scope / Mention for Bonus Points):**
    - Real-time collaborative editing of documents.
    - Voice-to-text input for queries.
    - Multi-modal support (asking questions about images/charts within the PDF).

---

3. Identifying Edge Cases

Edge cases are the "what ifs" that break naive systems. Identifying them early proves you have production-level experience, not just tutorial-level knowledge.

- **Data Size & Format Edge Cases:**
    - *"What if the user uploads a 1GB PDF?"* (Solution: Enforce file size limits, e.g., max 50MB, or use async background processing with a progress webhook).
    - *"What if the document is heavily scanned/image-based?"* (Solution: State that OCR is out of scope for V1, or requires a specialized vision model).
- **AI/LLM Specific Edge Cases:**
    - *"What if the user asks a question completely unrelated to the document?"* (Solution: The system should be designed to answer "I don't know" based on the retrieved context, rather than hallucinating).
    - *"What if the LLM API times out or returns a 500 error?"* (Solution: Implement graceful degradation, retry logic with exponential backoff, or fallback to a cheaper/faster model).

- **Security & Compliance Edge Cases (Your Superpower):**
    - *"What if the user uploads a document containing PII (Personally Identifiable Information) or PHI (Protected Health Information)?"* (Solution: Mention a PII redaction step in the ingestion pipeline before embedding).
    - *"What if a user tries prompt injection to bypass system instructions?"* (Solution: Mention LLM guardrails or input validation layers).

Search Keywords for Further Study

Copy and paste these into Google or YouTube to find high-quality examples:

- "how to gather functional requirements system design interview"
- "system design clarifying questions examples"
- "functional vs non functional requirements system design"
- "edge cases in system design interview LLM RAG"

💡 **AI Context & Interview Pro-Tip**

Because of your background in **healthcare data (2B+ entries)** and **cybersecurity (cybreal.net)**, you have a massive advantage in this section.

How to stand out: When the interviewer gives you a prompt like *"Design an AI Customer Support Agent,"* proactively ask a clarifying question that highlights your expertise:

"Before we define the core use cases, I want to clarify the data sensitivity. Will this agent handle PII or regulated data like healthcare records? If so, I'd like to include a PII-masking functional requirement before any data hits the external LLM API."

This single sentence instantly elevates you from a "junior candidate who knows LangChain" to a "senior engineer who builds secure, production-ready systems."

****

****

Non-Functional Requirements (NFR)

While Functional Requirements define *what* the system does, **Non-Functional Requirements define *how well* the system does it.** In interviews, NFRs are where you prove you can build a system that survives the real world.

---

1. Scalability: Handling Growth in Users/Data

Scalability is the system's ability to handle increased load (more users, more data, more requests) without a significant drop in performance.

- **Vertical Scaling (Scaling Up):** Adding more power (CPU, RAM, GPU) to an existing machine.
    - *Pros:* Simple, no code changes.
    - *Cons:* Hardware limits, single point of failure, expensive.
- **Horizontal Scaling (Scaling Out):** Adding more machines/nodes to a resource pool.
    - *Pros:* Theoretically infinite scale, better fault tolerance.
    - *Cons:* Requires stateless design, complex distributed systems management.
- **AI Context:**
    - *Data Scale:* Mention how you handled 2B+ healthcare records (e.g., "For vector storage scaling, I would use horizontal partitioning/sharding in pgvector or a managed service like Pinecone").
    - *Compute Scale:* LLM inference requires GPUs. Horizontal scaling of inference servers (e.g., using Kubernetes with vLLM for continuous batching) is critical to handle traffic spikes.

---

2. Availability: Understanding the "Nines"

Availability is the percentage of time the system is operational and accessible to users. It is measured in "nines."

- **The "Nines" Breakdown:**
    - **99% (Two nines):** ~3.65 days of downtime per year. (Acceptable for internal, non-critical tools).

    - **99.9% (Three nines):** ~8.76 hours of downtime per year. (Standard for most SaaS applications).
    - **99.99% (Four nines):** ~52.6 minutes of downtime per year. (Required for critical enterprise/healthcare systems).
    - **99.999% (Five nines):** ~5.26 minutes of downtime per year. (Telecom, financial trading, life-critical systems).
- **How to Achieve High Availability:** Eliminate single points of failure (SPOF) using load balancers, multi-AZ (Availability Zone) deployments, and database replication.
- **AI Context:** LLM APIs (like OpenAI or Bedrock) can experience outages. To achieve 99.9%+ availability for an AI app, you must design an **LLM Gateway with fallback routing** (e.g., if GPT-4 times out, automatically route the request to a locally hosted Llama-3 model).

---

3. Latency vs. Throughput

These two metrics are often confused but represent fundamentally different performance characteristics. You must know how to balance them.

- **Latency:** The time it takes to process a *single* request and return a response (measured in milliseconds or seconds).
    - *Goal:* Keep it low. Users abandon apps if latency is high.
- **Throughput:** The number of requests the system can process *concurrently* in a given time frame (measured in Requests Per Second - RPS, or Tokens Per Second - TPS).
    - *Goal:* Keep it high to maximize resource utilization.
- **The Trade-off:** Often, optimizing for one hurts the other. For example, adding heavy encryption or complex data validation increases latency but might be required for security.
- **AI Context:**
    - LLMs are inherently high-latency. You must design for **Time To First Token (TTFT)** using **streaming responses** (Server-Sent Events or WebSockets) so the user sees immediate progress, even if the total generation takes 5 seconds.

    - To improve throughput, use **batching** (grouping multiple user prompts into a single GPU forward pass) and **semantic caching** (returning a cached response if a user asks the exact same question).

---

4. Reliability & Security: Fault Tolerance and Data Protection

Reliability ensures the system performs correctly under stress, while security ensures it protects data from malicious actors.

- **Reliability / Fault Tolerance:**
    - **Graceful Degradation:** If a non-critical service (e.g., the "document summarization" feature) fails, the core feature (e.g., "basic chat") should still work.
    - **Retry Mechanisms:** Using exponential backoff and jitter when calling external LLM APIs to avoid thundering herd problems.
    - **Idempotency:** Ensuring that if a network timeout causes a client to retry a request (like "generate invoice"), the system doesn't process it twice.
- **Security:**
    - **Encryption:** Data in transit (TLS/HTTPS) and data at rest (AES-256 for database/storage).
    - **Access Control:** RBAC (Role-Based Access Control) and principle of least privilege.
- **AI Context (Your Superpower):**
    - *Healthcare Data:* Mention **HIPAA compliance**. Data must be encrypted at rest, and access logs must be immutable.
    - *Cybersecurity:* Proactively mention **PII/PHI redaction** *before* data is sent to a third-party LLM. Mention **LLM Guardrails** to prevent prompt injection attacks (e.g., a user trying to trick the AI into revealing its system prompt or accessing other users' data).

💡 Interview Pro-Tip: The "NFR First" Strategy

- When the interviewer gives you a prompt (e.g., *"Design a RAG system for a hospital"*), do not wait for them to ask about NFRs. **Volunteer them immediately** after clarifying Functional Requirements.
- Say something like:
- *"Since this is for a healthcare environment, I will assume we need **four nines (99.99%) of availability**, strict **HIPAA compliance** (meaning all data must be encrypted at rest and PII must be masked before hitting the LLM), and we need to optimize for **low Time-To-First-Token (TTFT)** via streaming, even if it means slightly lower overall throughput."*
- This immediately signals to the interviewer that you are a senior-level thinker who understands the real-world constraints of AI systems, leveraging your unique background in cybersecurity and large-scale healthcare data.

****

System Components

In High-Level Design, system components are the "Lego blocks" of your architecture. Your goal is not to know every detail of how they work internally, but to know **what they are, when to use them, and how they connect**.

---

1. Client (The Frontend / Entry Point)

The client is the interface through which the user (or another system) interacts with your application. It is responsible for presentation and capturing user input.

- **Web Browsers:** Single Page Applications (SPAs) like React, Next.js, or Vue. They communicate with the backend via HTTP/REST, GraphQL, or WebSockets.
- **Mobile Apps:** Native (iOS/Android) or cross-platform (React Native, Flutter). They often require optimized payloads to save battery and bandwidth.
- **IoT Devices:** Edge devices (sensors, cameras) that send telemetry data. They often use lightweight protocols like MQTT instead of HTTP.
- 💡 **AI Context:** For AI applications (like a ChatGPT clone), the client must support **streaming responses** (via Server-Sent Events or WebSockets) to display tokens one by one, reducing the perceived latency of the LLM.

---

2. Backend (The Brain / Business Logic)

The backend is the server-side application that processes requests, executes business logic, orchestrates workflows, and interacts with databases or external services.

- **API Servers:** Handle incoming HTTP requests, validate input, and route them to the appropriate service (e.g., Node.js, Python/FastAPI, Go, Java/Spring).
- **Business Logic Layer:** The core rules of the application (e.g., "Calculate the total price," "Verify user has permission to view this document").
- **Background Workers:** Asynchronous processes that handle long-running tasks without blocking the user (e.g., Celery, RabbitMQ consumers).
- 💡 **AI Context:** In an AI system, the backend acts as the **Orchestrator**. It receives the user prompt, fetches relevant chunks from the Vector DB, constructs the final prompt, calls the LLM, and streams the result back. Frameworks like LangGraph or CrewAI often live here.

---

3. Database (Persistent Storage)

Databases store the application's data. In HLD, you only need to understand the high-level trade-offs between the two main categories to make the right architectural choice.

- **Relational Databases (SQL):**
    - **Examples:** PostgreSQL, MySQL, Oracle.
    - **Characteristics:** Structured data, strict schema (schema-on-write), strong ACID compliance (Atomicity, Consistency, Isolation, Durability).
    - **Best For:** Transactional data, user profiles, billing information, and relational metadata (e.g., mapping a user_id to a document_id).
- **Non-Relational Databases (NoSQL):**
    - **Examples:** MongoDB (Document), Cassandra (Wide-column), Redis (Key-Value), Pinecone/Milvus (Vector).
    - **Characteristics:** Flexible schema (schema-on-read), highly scalable horizontally, often favors eventual consistency over strict ACID.
    - **Best For:** Unstructured data, high-velocity write operations, caching, and **AI workloads** (storing and querying high-dimensional embedding vectors).
- 💡 **AI Context:** A production AI system almost always uses **both**. You might use PostgreSQL to store user chat history and document metadata, and a Vector Database (like pgvector or Pinecone) to store the actual document embeddings for semantic search.

---

4. External Services (Third-Party Dependencies)

These are systems outside your direct control that your architecture relies on to function.

- **Examples:**
    - *Payments:* Stripe, PayPal.
    - *Authentication:* Auth0, AWS Cognito.
    - *Cloud Infrastructure:* AWS S3 (object storage), AWS Bedrock / OpenAI (LLM APIs), SendGrid (email).

- **Architectural Implications:** Because you don't control them, you must design for their failures. This means implementing **timeouts, retries with exponential backoff, circuit breakers, and fallback mechanisms**.
- 💡 **AI Context:** When using external LLM APIs (like OpenAI), you must design an **LLM Gateway** in your backend to handle rate limiting, cost tracking, and fallback routing (e.g., if OpenAI is down, route to a local Llama-3 model). Given your cybersecurity background, you must also ensure no PII/PHI is sent to these external services without prior redaction.

💡 **Interview Pro-Tip: The "Component Justification" Rule**

In an interview, never just draw a box labeled "Database." Always label it with the **specific technology and the "why"**.

Weak: "I will use a database to store the data." **Strong:** "I will use **PostgreSQL** to store user metadata and chat history because we need strong ACID compliance for billing and user relationships. However, for the document embeddings, I will use **pgvector** (or Pinecone) because it is optimized for high-dimensional similarity search (HNSW indexing), which a traditional SQL database cannot handle efficiently."

This demonstrates that you understand not just *what* the components are, but *how* to apply them to solve specific AI engineering problems.

****

Client-Server Architecture

What is Client-Server Architecture?

Client-Server architecture is the fundamental computing model where the workload is partitioned between two types of components: **Clients** (which request services or resources) and **Servers** (which provide them). In system design interviews, understanding this model is crucial for defining how data flows from the user's device to your backend infrastructure.

---

1. The Basic Request-Response Model

This is the standard communication pattern used in most web and mobile applications (primarily over HTTP/HTTPS).

- **The Client's Role:** Initiates the communication by sending a "Request" to a specific server endpoint (URL). The request includes a method (GET, POST, etc.), headers (authentication, content type), and optionally a body (payload).
- **The Server's Role:** Listens for incoming requests, processes the business logic, interacts with databases or other services, and sends back a "Response."
- **The Response:** Contains a Status Code (e.g., 200 OK, 404 Not Found, 500 Internal Server Error), headers, and the requested data (usually in JSON, XML, or HTML format).
- **Synchronous Nature:** In a basic request-response model, the client sends the request and *blocks* (waits) until the server finishes processing and returns the response.
- 💡 **AI Context:** LLM generation takes time (often 2–10 seconds). If you use a strict synchronous request-response model, the user stares at a blank screen. This is why AI apps use **Streaming** (via Server-Sent Events or WebSockets) to modify this model, sending tokens back to the client one by one as they are generated.

---

2. Stateful vs. Stateless Communication

This is a critical concept for scalability. It defines whether the server "remembers" the client between requests.

- **Stateful Communication:**
    - **Concept:** The server maintains the "state" (context, session data, user history) across multiple requests.

    - **Example:** Traditional banking apps where you log in once, and the server keeps a session ID in its memory to know who you are for the next 10 minutes.
    - **Pros:** Easier to implement complex, multi-step workflows on the server side.
    - **Cons: Hard to scale.** If you add a second server (Load Balancing), the second server doesn't know the user's state. You must implement "Sticky Sessions" or a centralized session store (like Redis), which adds complexity and a potential point of failure.
- **Stateless Communication:**
    - **Concept:** The server treats *every single request* as an independent, brand-new transaction. It does not store any client context between requests.
    - **How it works:** All necessary state must be included in the request itself (e.g., passing a JWT (JSON Web Token) in the header of every API call).
    - **Pros: Highly scalable.** Any server in a server farm can handle any request. If a server crashes, another can instantly take over without losing session data.
    - **Cons:** Requests are slightly larger (because they carry authentication tokens), and the server must re-validate the token on every request.
- 💡 **AI Context: LLM APIs are inherently stateless.** When you call OpenAI or AWS Bedrock, you must send the *entire* conversation history (or a summarized context) in every single request. The LLM does not "remember" your previous prompt. Designing your backend to manage this state (storing chat history in a DB and injecting it into the LLM prompt) is a core AI engineering task.

---

3. Thin Client vs. Thick (Fat) Client

This classification is based on how much processing power and logic is handled by the client device versus the server.

- **Thin Client:**
    - **Concept:** The client device does minimal processing. It relies almost entirely on the server for business logic, data processing, and storage. It primarily acts as an input/output terminal.
    - **Examples:** Standard web browsers (accessing a SaaS app), Chromebooks, terminal emulators.

    - **Pros:** Easy to update (just update the server), requires low-end hardware, highly secure (no data stored locally).
    - **Cons:** Requires a constant network connection, highly dependent on server latency.
- **Thick (Fat) Client:**
    - **Concept:** The client device handles significant processing, business logic, and local storage independently of the server.
    - **Examples:** Desktop applications (Photoshop, VS Code), native mobile apps with offline modes, PC video games.
    - **Pros:** Can work offline, highly responsive (no network latency for local actions), can leverage local hardware (GPU, camera, sensors).
    - **Cons:** Harder to deploy updates (users must download new versions), requires more powerful local hardware, harder to secure local data.
- 💡 **AI Context:**
    - *Thin Client AI:* A standard web-based ChatGPT clone. The browser just renders text; the heavy lifting (inference) happens on the server's GPUs.
    - *Thick Client AI:* Running a local AI assistant (like Ollama or LM Studio) on your laptop. The client machine uses its own local GPU/RAM to run the LLM, requiring zero server infrastructure and offering total data privacy.

💡 **Interview Pro-Tip: The "Stateless by Default" Rule**

If an interviewer asks you how to design the backend for a new AI application, **always default to a stateless architecture** unless there is a highly specific reason not to.

How to phrase it in an interview:

"I will design the API layer to be completely stateless to ensure we can horizontally scale our inference servers behind a load balancer. Any required state, such as user chat history or document metadata, will be externalized and stored in a fast, centralized database like PostgreSQL or Redis, rather than kept in the application server's memory."

This shows the interviewer that you inherently understand how to build systems that can scale to millions of users, which is exactly what they are looking for in an AI Engineer.

Request-Response Lifecycle

The Classic Question: "What happens when you type a URL into a browser?"

Interviewers ask this to see if you understand the journey of a network packet from a user's device to your backend infrastructure and back. It proves you know how the internet actually works under the hood.

---

1. DNS Resolution (Finding the Server)

Before any data can be sent, the browser needs to know the IP address of the server hosting the domain (e.g., www.chatapp.com).

- **The Process:**
    1. **Browser Cache:** Checks if it already knows the IP.
    2. **OS Cache:** Checks the operating system's cache.
    3. **Router/ISP:** Checks the local router, then asks the Internet Service Provider's DNS server.
    4. **Recursive Search:** If the ISP doesn't know, it queries Root, Top-Level Domain (TLD), and finally the Authoritative Name Servers to get the exact IP address.
- **Key Takeaway:** DNS is essentially the "phonebook" of the internet.

2. TCP Handshake (Establishing the Connection)

Once the IP is found, the browser must establish a reliable connection with the server using the Transmission Control Protocol (TCP).

- **The 3-Way Handshake:**
    1. **SYN:** The client sends a "synchronize" packet to the server to initiate a connection.
    2. **SYN-ACK:** The server receives it and sends back a "synchronize-acknowledge" packet.
    3. **ACK:** The client acknowledges the server's response. The connection is now established.
- **Key Takeaway:** TCP ensures that data packets arrive in order and without errors.

3. TLS/SSL Handshake (Securing the Connection)

If the URL is https://, the connection must be encrypted using Transport Layer Security (TLS), the successor to SSL.

- **The Process:**
    1. **Client Hello:** The browser sends supported encryption algorithms and a random number.
    2. **Server Hello:** The server responds with its SSL Certificate, chosen algorithm, and its own random number.
    3. **Verification:** The browser verifies the server's certificate with a trusted Certificate Authority (CA).
    4. **Key Exchange:** Both sides generate a shared "session key" to encrypt all subsequent data.
- **Key Takeaway:** This step guarantees that the data sent between the client and server cannot be intercepted or read by hackers.

4. Load Balancer (Distributing the Traffic)

The encrypted request now hits your infrastructure's front door.

- **The Process:** The Load Balancer (LB) receives the incoming request and decides which backend server should handle it based on algorithms (e.g., Round Robin, Least Connections) and server health checks.
- **Key Takeaway:** The LB ensures no single server is overwhelmed and provides high availability (if one server dies, the LB routes traffic to the others).

5. Web Server / Reverse Proxy (The Gatekeeper)

Often sitting right behind the Load Balancer (sometimes combined into one, like Nginx).

- **The Process:**
    - It handles **SSL Termination** (decrypting the HTTPS traffic so the backend doesn't have to).
    - It serves **static assets** directly (images, CSS, JS) to save the application servers from doing unnecessary work.
    - It forwards dynamic requests (like API calls) to the Application Servers.

6. Application Server (The Brains)

This is where your actual business logic lives (e.g., a Python/FastAPI, Node.js, or Java/Spring Boot application).

- **The Process:**
    - It parses the HTTP request (URL, headers, body).
    - It executes the required business logic (e.g., "Verify user is logged in," "Calculate shopping cart total").
    - It interacts with databases or external APIs to fetch or save data.
- **Key Takeaway:** This is the core of your microservices or monolithic architecture.

7. Database (The Memory)

The Application Server queries the database to get the data needed to fulfill the request.

- **The Process:** The app server sends a query (e.g., SELECT * FROM users WHERE id = 123), the database processes it using indexes, retrieves the data, and sends it back to the app server.
- **The Return Journey:** The App Server formats the data (usually into JSON), sends it back through the Web Server → Load Balancer → Internet → Browser, which renders the UI.

Search Keywords for Further Study

- "what happens when you type a url in browser system design"
- "http request lifecycle dns to database"
- "TCP 3 way handshake explained"
- "TLS handshake step by step"
- "reverse proxy vs load balancer system design"

💡 **AI Context: Applying the Lifecycle to LLM Systems**

In an AI Engineer interview, you must map this traditional lifecycle to an AI workload. When a user sends a prompt to your RAG system, it travels this exact path, but with AI-specific optimizations:

- **Where to add Caching:**

    - *CDN/Edge:* Cache the frontend static assets.
    - *API Gateway / Semantic Cache:* If a user asks a question that has been asked before, intercept the request at the API Gateway and return the cached LLM response immediately, bypassing the expensive inference servers.
- **Where to add Rate Limiting:**
    - Implement strict rate limiting at the **Load Balancer / API Gateway** level. LLM tokens cost money. You must prevent malicious users or buggy clients from sending infinite requests and bankrupting your cloud bill.
- **Handling High Latency (The AI Twist):**
    - Because LLM inference takes 2–10 seconds, a standard synchronous HTTP response will cause the browser to time out or the user to think the app froze.
    - *Solution:* The Application Server must switch to **Streaming (Server-Sent Events - SSE)**. It sends the HTTP headers back immediately, and then streams the generated tokens back to the client one by one over the same TCP connection.
- **The AI Data Flow:**
    - Client → LB → API Gateway → **Orchestrator Service** (App Server) → **Vector DB** (Retrieval) → **LLM Inference Server** (Generation) → Stream back to Client.

💡 **Interview Pro-Tip: The "Zoom In" Technique**

If asked this question, don't just list the steps robotically. Give a high-level summary first, and then **ask the interviewer where they want to zoom in.**

How to phrase it:

"At a high level, the request goes through DNS resolution, establishes a TCP/TLS connection, hits our Load Balancer, passes through a Reverse Proxy, hits the Application Server, and finally queries the Database. Since this is an AI role, I'd specifically highlight that at the Load Balancer/API Gateway layer, we would implement semantic caching and rate limiting to protect our expensive LLM endpoints. Would you like me to deep-dive into the networking protocols, or focus on how the Application Server orchestrates the LLM call?"

This shows you know the basics perfectly but are ready to pivot to advanced AI architecture.

****

Monolithic Architecture

Understanding monoliths is just as important as understanding microservices, because a common interview trap is assuming "microservices are always better." Senior engineers know when a monolith is actually the *right* choice.

---

1. Definition and Characteristics

A monolithic architecture is a traditional software design model where the entire application is built as a single, unified unit.

- **Single Codebase:** The user interface, business logic, and data access layers all reside in the same code repository.
- **Tightly Coupled:** Components communicate with each other via in-memory function or method calls, not over a network.
- **Single Deployment Artifact:** The entire application is compiled and deployed as one unit (e.g., a single .jar file, a single Python package, or one Docker container).
- **Shared Resources:** All components share the same memory space, database, and compute resources (CPU/RAM/GPU).

---

2. Pros of Monolithic Architecture

Do not dismiss monoliths. They have distinct advantages, especially in the early stages of a product or for specific workloads.

- **Simplicity in Development & Testing:**
    - Developers can run the entire application locally with a single command.
    - End-to-end testing is straightforward because there are no network boundaries or distributed state to mock.
- **Simplified Deployment:**
    - You only have one artifact to build, version, and deploy. No need for complex orchestration tools like Kubernetes or service meshes early on. PV* **Low Network Latency:**

    - Since components communicate via in-memory function calls rather than HTTP/gRPC over a network, there is zero network overhead. This makes monoliths inherently fast for tightly coupled operations.

- **Easier Data Management:**
    - A single database means you don't have to worry about distributed transactions, eventual consistency, or complex data synchronization across services.

---

3. Cons of Monolithic Architecture

As an application grows in users, data, and team size, the monolith begins to show severe limitations.

- **Inefficient Scaling (The "All or Nothing" Problem):**
    - You cannot scale individual components. If your document ingestion module is under heavy load, you must spin up another instance of the *entire* monolith, including the UI and LLM inference components. This is highly inefficient and expensive (especially when GPUs are involved).
- **Single Point of Failure (SPOF):**
    - A bug or memory leak in one minor module (e.g., a faulty PDF parsing library) can crash the entire application, taking down the user-facing chat and all other features with it.
- **Technology Lock-in:**
    - The entire application is tied to a single technology stack. If you want to use Rust for a high-performance vector search module, you can't easily do it without rewriting the whole app or creating awkward foreign function interfaces (FFI).
- **Slow Build and Deployment Cycles:**
    - As the codebase grows, build times increase. A minor text change requires rebuilding and redeploying the entire massive application, increasing the risk of downtime and slowing down developer velocity.
- **Onboarding Complexity:**
    - New developers must understand the entire massive codebase to make even small changes, leading to a steep learning curve.

💡 AI Context: Monoliths in AI System Design

In AI engineering, the decision to use a monolith vs. microservices often comes down to **compute heterogeneity** (different parts of the system need different hardware).

- **When a Monolith is the RIGHT Choice for AI:**
    - *Early-stage startups or MVPs:* If you are building a B2C AI app with < 5,000 users, a "Modular Monolith" (e.g., a single FastAPI application handling routing, basic RAG, and database calls) is the best choice. It allows you to iterate rapidly without DevOps overhead.
    - *Edge AI / Local AI:* Running an LLM locally on a user's device (Thick Client) is inherently a monolithic design, and that's perfectly fine for privacy and offline use cases.
- **When a Monolith is the WRONG Choice for AI:**
    - *Divergent Scaling Needs:* Imagine your app has two features: **Batch Document Ingestion** (CPU-heavy, asynchronous, high memory) and **Real-time Chat** (GPU-heavy, latency-sensitive). If these are in a monolith, a spike in document uploads will consume all the RAM/CPU, causing the real-time chat to slow down or crash.
    - *Solution:* You must decouple them. The ingestion pipeline should be a separate, CPU-optimized microservice (using Kafka/Celery), while the chat inference runs on a separate, GPU-optimized service.

💡 Interview Pro-Tip: The "Monolith First" Principle

Interviewers love candidates who show pragmatic maturity. Do not automatically say, *"I will use microservices because monoliths are bad."*

Instead, use this powerful framing:

*"For the V1 of this system, I would recommend starting with a **well-structured modular monolith**. This allows the team to move fast, keep deployment simple, and validate the product without the operational overhead of distributed systems. However, because I anticipate the **document ingestion** (CPU-bound) and **LLM inference** (GPU-bound) will have vastly different scaling requirements, I will design the codebase with clear internal boundaries. This way, when*

we hit scale, we can easily extract those modules into independent microservices using the ***Strangler Pattern*** *without a complete rewrite."*

Mentioning the **Strangler Pattern** (a technique to gradually migrate a monolith to microservices by replacing specific functionalities with new services) will instantly score you major points.

******

Microservices Architecture

1. Definition and Characteristics

Microservices architecture is an approach where a large, complex application is broken down into a suite of small, loosely coupled, and independently deployable services.

- **Independent Deployability:** Each service can be built, tested, deployed, and scaled independently without affecting the rest of the application.
- **Decentralized Data Management:** Unlike a monolith with a single shared database, each microservice typically owns its own database (Database-per-Service pattern) to ensure loose coupling.
- **Business-Centric Design:** Services are organized around specific business capabilities or domains (e.g., "User Service," "Billing Service," "Inference Service") rather than technical layers.
- **Smart Endpoints, Dumb Pipes:** The services contain the business logic ("smart"), while the communication mechanisms like message queues or simple REST APIs ("dumb pipes") just pass data without complex routing logic.

---

2. Pros of Microservices Architecture

Microservices solve the major scaling and organizational bottlenecks of monoliths.

- **Independent Scaling:** You can scale only the services that need it. If your "Chat Inference" service is under heavy load but your "User Profile" service is idle, you can add more GPU nodes *only* to the inference service, saving massive cloud costs.
- **Technology Flexibility (Polyglot Programming):** Each service can use the best tool for its specific job. You can use **Python/FastAPI** for your ML/Embedding services, **Go or Rust** for high-throughput API gateways, and **Node.js** for real-time WebSocket chat, all within the same product.
- **Fault Isolation:** If a non-critical service crashes (e.g., the "Document Summarization" service throws an Out-Of-Memory error), the core "Chat" service remains up and running. The failure is contained.
- **Faster Development Cycles:** Large engineering teams can work on different services simultaneously without stepping on each other's toes or dealing with massive merge conflicts in a single codebase.

---

3. Cons of Microservices Architecture

Microservices introduce significant operational and architectural complexity. **Never choose microservices just because they are trendy.**

- **Network Overhead & Latency:** Services must communicate over the network (HTTP/gRPC) instead of in-memory function calls. This adds latency and requires robust handling of network timeouts and retries.
- **Distributed Tracing Complexity:** Debugging is much harder. If a user request fails, it might have passed through 5 different services. You *must* implement distributed tracing (e.g., OpenTelemetry, Jaeger) to track a single request ID across the entire system.
- **Data Consistency Challenges:** Because each service has its own database, you cannot use simple ACID transactions across multiple services. You must rely on **Eventual Consistency** and complex patterns like the **Saga Pattern** to manage distributed transactions.
- **Operational Overhead:** Requires mature DevOps practices. You need containerization (Docker), orchestration (Kubernetes), CI/CD pipelines, and potentially a Service Mesh (Istio) just to keep the system running.

---

4. Bounded Contexts (Domain-Driven Design Basics)

How do you decide where to draw the lines between microservices? You use **Bounded Contexts**, a core concept from Domain-Driven Design (DDD).

- **Definition:** A bounded context is a logical boundary within which a specific domain model is defined and applicable. Inside this boundary, a term (like "User" or "Document") has a single, unambiguous meaning.
- **The Problem it Solves:** In a massive monolith, a "User" entity might mean something different to the Billing team (credit cards, subscriptions) than to the Analytics team (clicks, session times). This leads to a tangled, fragile codebase.
- **The Microservices Solution:** You draw a boundary around the "Billing" domain and the "Analytics" domain. Each becomes its own microservice with its own "User" data model. They communicate via well-defined APIs, keeping the domains isolated and clean.

💡 AI Context: Why Microservices are Critical for AI Systems

In AI engineering, microservices are almost mandatory for production systems because AI workloads have wildly different hardware and scaling profiles.

- **Separating Compute Profiles:**
    - **Embedding/Ingestion Service:** This is highly **CPU-bound** and memory-intensive (parsing PDFs, running OCR, generating embeddings). It runs asynchronously in the background.
    - **Chat Inference Service:** This is highly **GPU-bound** and latency-sensitive. It runs synchronously to serve live user requests.
    - *The Win:* If you put these in a monolith, a user uploading a 500-page PDF will consume all the CPU/RAM, causing the live chat to lag or crash. By separating them into microservices, the ingestion spike has zero impact on chat latency.
- **Model Swapping:** If you want to upgrade your embedding model from text-embedding-ada-002 to a newer local model, you can deploy a new version of the "Embedding Service" and route traffic to it without touching the "Chat Service" or the database schema.

💡 Interview Pro-Tip: The "When to Use" Framework

Interviewers will test your maturity by asking: *"Should we use microservices for this new AI startup?"*

How to answer like a Senior Engineer:

*"Microservices are excellent for scaling, but they come with high operational overhead. I would only recommend microservices if we meet at least two of these criteria: **1)** We have multiple distinct teams working on different domains (Conway's Law), **2)** We have distinct scaling requirements (e.g., CPU-heavy ingestion vs. GPU-heavy inference), or **3)** We need independent deployment cycles to move fast. If we are an early-stage team building an MVP, I would strongly advocate for a **Modular Monolith** first, and refactor into microservices only when we hit actual scaling bottlenecks."*

****

Architectural Trade-offs

This is arguably the **most important topic in all of High-Level Design**. Interviewers do not care if you pick the "right" technology; they care if you can articulately explain *why* you picked it and *what you sacrificed* to get it.

---

1. The "No Perfect Architecture" Mindset

- **The Core Truth:** Every architectural decision is a compromise. There is no universally "best" database, framework, or pattern.
- **The Interview Goal:** Your job is not to find the perfect solution, but to find the *least worst* solution for the specific constraints of the problem (e.g., budget, team size, latency requirements, compliance).
- **The Golden Phrase:** Always start your justification with: *"It depends on our priorities. If we prioritize X, we should do Y, but the trade-off is Z."*

---

2. Consistency vs. Availability (Intro to CAP Theorem)

The CAP Theorem states that a distributed data store can only guarantee two of the following three properties at the same time: **C**onsistency, **A**vailability, and **P**artition Tolerance. (Since network partitions (P) are a reality in distributed systems, you must choose between C and A).

- **Consistency (C):** Every read receives the most recent write or an error. All nodes see the same data at the same time.
- **Availability (A):** Every request receives a (non-error) response, without the guarantee that it contains the most recent write.
- **The Trade-off:**
    - *Choose Consistency* when data accuracy is critical (e.g., financial transactions, user billing, or healthcare patient records). Trade-off: The system might reject requests or slow down during a network glitch.
    - *Choose Availability* when uptime is critical and stale data is acceptable (e.g., social media feeds, product recommendations, or semantic search results). Trade-off: A user might see slightly outdated data for a few seconds.
- 💡 **AI Context:** In a RAG system, your **Vector Database** can often be *eventually consistent* (Availability over Consistency). If a newly uploaded document takes 2 seconds

to appear in search results, it's acceptable. However, the **PostgreSQL database** storing user billing or access permissions must be *strongly consistent* (Consistency over Availability).

---

3. Latency vs. Throughput

These two metrics are fundamentally at odds with each other in system design.

- **Latency:** The time it takes to complete a *single* operation (e.g., milliseconds per request). Lower is better.
- **Throughput:** The number of operations the system can complete in a given time frame (e.g., Requests Per Second - RPS, or Tokens Per Second - TPS). Higher is better.
- **The Trade-off:** Optimizing for one often degrades the other.
    - *Example:* Processing requests one by one as they arrive yields excellent **latency** but terrible **throughput** (the GPU sits idle between requests).
    - *Example:* Waiting to collect 32 requests and processing them together (batching) yields excellent **throughput** but terrible **latency** for the first user in the batch.
- 💡 **AI Context:** LLM inference is the ultimate latency vs. throughput battleground.
    - To improve **throughput**, we use *continuous batching* (e.g., vLLM) to pack multiple user requests into a single GPU forward pass.
    - To improve perceived **latency**, we use *streaming* (Server-Sent Events) to send the Time-To-First-Token (TTFT) immediately, even if the total generation time is longer.

---

4. Build vs. Buy

This is the classic engineering dilemma: Do we build a custom solution in-house, or pay for a managed third-party service?

- **Build (In-House / Open Source):**
    - *Pros:* Total control over data, no vendor lock-in, highly customizable, potentially cheaper at massive scale.
    - *Cons:* High initial development time, requires dedicated DevOps/MLOps maintenance, you own the bugs and security patches.

- **Buy (Managed / SaaS):**
    - *Pros:* Extremely fast time-to-market, managed scaling, built-in security/compliance, less operational overhead.
    - *Cons:* Vendor lock-in, recurring costs that scale linearly (or exponentially) with usage, less control over underlying infrastructure.
- 💡 **AI Context (Your Superpower):**
    - *Vector DBs:* "For our MVP, I would **buy** Pinecone or Weaviate to move fast. However, given my experience with healthcare data compliance, at enterprise scale, I would **build** using PostgreSQL with pgvector. It keeps sensitive data within our own VPC, avoids third-party data leakage risks, and leverages our existing SQL infrastructure."
    - *LLM Hosting:* "We will **buy** managed inference via AWS Bedrock for base models to avoid GPU maintenance, but we will **build** our own LLM Gateway and semantic caching layer to control costs and route traffic intelligently."

💡 **Interview Pro-Tip: The "Trade-off Framework"**

When the interviewer asks, *"Why did you choose X over Y?"*, never just give a one-sentence answer. Use this 3-step framework to sound like a senior engineer:

1. **Acknowledge the alternative:** *"We could have used a managed service like Pinecone..."*
2. **State the primary driver for your choice:** *"...but given our strict healthcare data residency requirements (HIPAA) and my experience with pgvector..."*
3. **Explicitly state the trade-off you are accepting:** *"...I chose to build on PostgreSQL. The trade-off is that our team will need to manage index tuning and backups ourselves, but we gain absolute data sovereignty and avoid vendor lock-in."*

This proves you aren't just memorizing buzzwords; you are making deliberate, business-aware engineering decisions.